

Backpropagation and Neural Network Basics

Nguyễn Duy Tân *

EE HCMUT ML&IoT Lab,
Trường Đại học Bách khoa Thành phố Hồ Chí Minh,
Đại học Quốc gia Thành phố Hồ Chí Minh
`tan.nguyen231305@hcmut.edu.vn`

Thành phố Hồ Chí Minh, Tháng 7 Năm 2026

Abstract

Nội dung của tài liệu cung cấp một cái nhìn toàn diện về nền tảng toán học và cơ chế hoạt động thực tiễn của Mạng nơ-ron (Neural Networks). Về mặt lý thuyết, mạng nơ-ron hoạt động như những bộ phân loại có khả năng tạo ra các ranh giới quyết định phi tuyến tính, sử dụng cấu trúc nhiều lớp nơ-ron kết hợp cùng các hàm kích hoạt (như Sigmoid, Tanh, ReLU, hoặc các biến thể hiện đại như GELU, SwiGLU) để xấp xỉ các hàm toán học phức tạp. Quá trình tính toán thuận (feed-forward computation) chuyển đổi dữ liệu đầu vào thành các điểm số, sau đó được đánh giá sai số thông qua các hàm mất mát (loss functions) như Maximum Margin hoặc Cross-entropy. Để tối ưu hóa và cập nhật các trọng số, hệ thống sử dụng thuật toán lan truyền ngược (backpropagation), áp dụng linh hoạt quy tắc chuỗi (chain rule) và ma trận Jacobian trên các biểu đồ tính toán (computation graph) nhằm truyền ngược và tái sử dụng tín hiệu lỗi (gradients) một cách hiệu quả. Bên cạnh các cơ sở toán học khắt khe, tài liệu còn nhấn mạnh các kỹ thuật thực hành quan trọng để huấn luyện mạng nơ-ron thành công, bao gồm các thuật toán tối ưu hóa linh hoạt (như AdaGrad, RMSProp, Adam), phương pháp khởi tạo tham số (Xavier initialization), kỹ thuật điều chuẩn nhằm chống quá khớp (L2 regularization, Dropout), tiền xử lý dữ liệu, và kỹ thuật kiểm tra đạo hàm bằng số học (gradient check) để xác minh tính chính xác của các công thức giải tích.

1 Cơ sở toán học của Mạng Nơ-ron

Mạng nơ-ron nhân tạo (Artificial Neural Networks – ANNs) là một họ các mô hình học máy mạnh mẽ, được lấy cảm hứng từ cấu trúc xử lý thông tin của hệ thần kinh sinh học. Khác với các bộ phân loại tuyến tính (như hồi quy logistic hay SVM với nhân tuyến tính), mạng nơ-ron có khả năng tạo ra các ranh giới quyết định phi tuyến tính phức tạp, cho phép phân tách dữ liệu mà các mô hình tuyến tính không thể xử lý được [?, p.1]. Để đạt được điều này, mạng nơ-ron xây dựng các biểu diễn trung gian (hidden representations) thông qua việc xếp chồng nhiều lớp biến đổi phi tuyến, từ đó học được các đặc trưng trừu tượng và phù hợp với tác vụ. Phần này trình bày nền tảng toán học của mạng nơ-ron, bao gồm cấu trúc nơ-ron, các hàm kích hoạt, cách thức phân loại, và các hàm mất mát thường dùng.

1.1 Nơ-ron và Lớp Nơ-ron

Một nơ-ron nhân tạo là đơn vị tính toán cơ bản nhất, nhận một vector đầu vào $x \in \mathbb{R}^n$ và tạo ra một giá trị vô hướng (activation) a thông qua hai bước: (1) tổ hợp tuyến tính các đầu vào với trọng số, và (2) áp dụng một hàm kích hoạt phi tuyến f [?, p.1]. Cụ thể, với vector trọng số $w \in \mathbb{R}^n$ và độ lệch (bias) $b \in \mathbb{R}$, đầu ra của nơ-ron sigmoid (một trong những lựa chọn phổ biến) được tính như sau:

* Bài viết được viết lại từ các slides [?, ?] và sử dụng AI như một công cụ hỗ trợ.

$$a = \frac{1}{1 + \exp(-(w^T x + b))}.$$

Trong thực tế, để thuận tiện cho việc tính toán và biểu diễn dưới dạng ma trận, ta thường gộp bias vào vector trọng số bằng cách thêm một chiều hằng số 1 vào đầu vào: $\tilde{x} = [x; 1]$, $\tilde{w} = [w; b]$, khi đó $w^T x + b = \tilde{w}^T \tilde{x}$.

Khi một đầu vào x được đưa đồng thời vào m nơ-ron khác nhau (một lớp), ta có thể véc-tơ hóa toàn bộ quá trình. Gọi $W \in \mathbb{R}^{m \times n}$ là ma trận trọng số, mỗi hàng W_i tương ứng với trọng số của nơ-ron thứ i , và $b \in \mathbb{R}^m$ là vector bias. Khi đó, phép biến đổi tuyến tính cho toàn bộ lớp là:

$$z = Wx + b,$$

với $z \in \mathbb{R}^m$ là vector các đầu vào tuyến tính (pre-activation) của từng nơ-ron. Sau đó, áp dụng hàm kích hoạt f theo từng phần tử (element-wise) để thu được vector activation:

$$a = f(z) = [f(z_1), f(z_2), \dots, f(z_m)]^T.$$

Như vậy, một lớp nơ-ron về cơ bản thực hiện một phép biến đổi affine ($Wx + b$) rồi đi qua một hàm phi tuyến f [?, p.2]. Chính sự kết hợp này, khi được xếp chồng nhiều lớp, tạo nên sức mạnh biểu diễn của mạng nơ-ron sâu.

1.2 Vai trò của Hàm Kích hoạt Phi tuyến

Một câu hỏi quan trọng đặt ra là: tại sao cần có hàm kích hoạt phi tuyến? Nếu không có chúng, toàn bộ mạng dù có bao nhiêu lớp cũng chỉ tương đương với một phép biến đổi tuyến tính duy nhất, vì:

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2) = W'x + b'.$$

Điều này có nghĩa là một mạng toàn tuyến tính không thể học được các mối quan hệ phức tạp trong dữ liệu, và do đó không thể giải quyết các bài toán thực tế như nhận dạng ảnh hay xử lý ngôn ngữ tự nhiên [?, p.17]. Chính các hàm kích hoạt phi tuyến (như sigmoid, tanh, ReLU, ...) đã cung cấp khả năng xấp xỉ bất kỳ hàm liên tục nào khi có đủ số lượng nơ-ron và lớp (theo định lý xấp xỉ phổ dụng – Universal Approximation Theorem). Nhờ đó, mạng nơ-ron có thể tạo ra các ranh giới quyết định phi tuyến rất linh hoạt, như minh họa trong Hình 1 của tài liệu [?, p.1].

1.3 Các Hàm Kích hoạt Phổ biến

Lựa chọn hàm kích hoạt ảnh hưởng trực tiếp đến tốc độ hội tụ và chất lượng của mô hình. Dưới đây là một số hàm thường được sử dụng:

- **Sigmoid (Logistic):** $\sigma(z) = \frac{1}{1+e^{-z}}$, có miền giá trị $(0, 1)$, thường dùng ở tầng đầu ra để biểu diễn xác suất. Đạo hàm $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Tuy nhiên, sigmoid dễ bị bão hòa (saturation) khi $|z|$ lớn, làm cho gradient tiến gần 0 và gây khó khăn cho việc huấn luyện sâu [?, p.12].
- **Tanh:** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1$, miền giá trị $(-1, 1)$. Tanh thường hội tụ nhanh hơn sigmoid vì có trung bình bằng 0, đạo hàm $\tanh'(z) = 1 - \tanh^2(z)$ [?, p.13].
- **ReLU (Rectified Linear Unit):** $f(z) = \max(0, z)$. Đây là lựa chọn phổ biến nhất cho các mạng sâu nhờ tính đơn giản, huấn luyện nhanh, và tránh được hiện tượng gradient biến mất (vanishing gradient) ở vùng dương. Nhược điểm là các nơ-ron có $z < 0$ sẽ bị "chết" (dead ReLU) và không bao giờ được kích hoạt lại [?, p.15].
- **Các biến thể hiện đại:** Gần đây, các hàm như GELU (Gaussian Error Linear Unit), SiLU (Swish), GLU (Gated Linear Unit) và SwiGLU được sử dụng rộng rãi trong các kiến trúc Transformer và LLM (ví dụ Llama, Qwen) vì mang lại hiệu suất cao hơn [?, p.16]. Ví dụ, SwiGLU được định nghĩa là $\text{SwiGLU}(x) = (xV + c) \otimes \text{Swish}_\beta(xW + b)$, trong đó $\otimes$ là tích Hadamard và $\text{Swish}_\beta(x) = x \cdot \sigma(\beta x)$.

Việc chọn hàm kích hoạt phù hợp là một siêu tham số quan trọng, thường được xác định qua thực nghiệm.

1.4 Phân loại với Mạng Nơ-ron

Trong các bài toán phân loại, mạng nơ-ron thường được sử dụng với một kiến trúc bao gồm: một tầng embedding (để chuyển các chỉ số từ từ vựng thành vector liên tục), một hoặc nhiều tầng ẩn với các hàm kích hoạt, và một tầng đầu ra với hàm softmax (cho phân loại đa lớp) hoặc sigmoid (cho phân loại nhị phân). Điểm khác biệt then chốt so với các mô hình tuyến tính truyền thống là mạng nơ-ron *học đồng thời* cả ma trận trọng số của tầng phân loại W và các biểu diễn phân bố (distributed representations) của từ vựng [?, p.13]. Cụ thể, nếu đầu vào là một chỉ số từ e (one-hot), thì tầng embedding sẽ biến đổi thành vector $x = Le$, với L là ma trận embedding học được. Sau đó, vector này được đưa qua các tầng ẩn để tạo ra các biểu diễn trừu tượng hơn, và cuối cùng tầng softmax cho ra xác suất trên các lớp.

Quá trình này có thể được biểu diễn bằng các phép toán ma trận. Ví dụ, với một mạng một tầng ẩn, đầu ra điểm số (score) cho một cửa sổ ngữ cảnh được tính như sau [?, p.3]:

$$z = Wx + b, \quad a = f(z), \quad s = U^T a,$$

trong đó x là vector đầu vào (ghép các embedding của các từ trong cửa sổ), W, b là tham số của tầng ẩn, U là ma trận trọng số của tầng đầu ra, và s là điểm số (thường được đưa qua softmax để thành xác suất). Mô hình này có thể phân loại một từ trung tâm (ví dụ: nhận dạng thực thể có tên) dựa trên ngữ cảnh xung quanh nó [?, p.10].

1.5 Hàm Mất mát cho Huấn luyện

Để huấn luyện mạng nơ-ron, ta cần một hàm mất mát (loss function) đo lường sai số giữa dự đoán của mô hình và nhãn thực tế. Hai loại hàm mất mát phổ biến trong phân loại là:

- **Hàm mất mát Maximum Margin (Marginal Loss):** Thường được sử dụng với các mô hình dựa trên điểm số (ví dụ: trong SVM hoặc một số kiến trúc NER). Ý tưởng là đảm bảo điểm số của mẫu đúng s phải lớn hơn điểm số của mẫu sai s_c một khoảng cách an toàn Δ (thường chọn $\Delta = 1$). Hàm mất mát cho một cặp (đúng, sai) là [?, p.4]:

$$J = \max(0, \Delta + s_c - s).$$

Hàm này chỉ phát sinh lỗi khi $s_c + \Delta > s$, tức là mô hình chưa phân tách đủ tốt hai lớp.

- **Hàm mất mát Cross-Entropy (Entropy chéo):** Đây là hàm mất mát tiêu chuẩn cho phân loại đa lớp khi đầu ra mô hình là một phân phối xác suất (thường từ softmax). Với phân phối đích p (thường là one-hot) và phân phối dự đoán q , cross-entropy được định nghĩa là [?, p.18]:

$$H(p, q) = - \sum_{c=1}^C p(c) \log q(c).$$

Khi p là one-hot tại lớp y , biểu thức này rút gọn thành $-\log q(y)$, chính là âm log-likelihood của lớp đúng. Cross-entropy thường được ưa chuộng hơn trong các bài toán học sâu vì cho gradient ổn định và dễ tối ưu.

Cả hai hàm mất mát đều có thể được tối thiểu hóa bằng các thuật toán dựa trên gradient, mà nền tảng của chúng là phép tính đạo hàm và thuật toán lan truyền ngược (backpropagation) sẽ được trình bày trong các phần tiếp theo.

Tóm lại, mạng nơ-ron cung cấp một khung toán học thống nhất để xây dựng các bộ phân loại phi tuyến, với khả năng học biểu diễn dữ liệu một cách linh hoạt. Việc nắm vững cấu trúc nơ-ron, vai trò của hàm kích hoạt, và các hàm mất mát là tiền đề để hiểu sâu hơn về cách thức huấn luyện mạng thông qua backpropagation và các kỹ thuật tối ưu hóa hiện đại.

2 Tính toán Đạo hàm và Ma trận

Trong phần trước, chúng ta đã làm quen với cấu trúc của mạng nơ-ron và các hàm mất mát. Để huấn luyện mạng, chúng ta cần tối ưu hóa hàm mất mát theo các tham số (trọng số và độ lệch) bằng cách sử dụng các phương pháp dựa trên gradient, như Gradient Descent hoặc các biến thể của nó. Điều này đòi hỏi chúng ta phải tính được đạo hàm (gradient) của hàm mất mát đối với từng tham số. Đối với các mạng nơ-ron hiện đại với hàng triệu tham số, việc tính toán thủ công từng đạo hàm riêng là bất khả thi. May mắn thay, chúng ta có thể sử dụng công cụ toán học mạnh mẽ là **phép tính ma trận (matrix calculus)** và **quy tắc chuỗi (chain rule)** để tính toán một cách có hệ thống và hiệu quả tất cả các đạo hàm cần thiết [?, p.20]. Phần này sẽ giới thiệu các khái niệm cốt lõi của ma trận Jacobian, cách áp dụng quy tắc chuỗi trong bối cảnh đa biến, và một số quy ước hình dạng (shape convention) quan trọng khi thực hiện các phép toán này trong thực tế.

2.1 Đạo hàm, Gradient và Ma trận Jacobian

Đối với một hàm số một biến $f(x)$ có một đầu vào và một đầu ra, đạo hàm $\frac{df}{dx}$ cho biết tốc độ thay đổi của f khi x thay đổi một lượng nhỏ. Khi hàm có nhiều đầu vào $f(x_1, x_2, \dots, x_n)$, gradient của nó là một vector chứa tất cả các đạo hàm riêng phần:

$$\nabla f = \frac{\partial f}{\partial x} = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right].$$

Tuy nhiên, trong mạng nơ-ron, chúng ta thường làm việc với các hàm có *nhiều đầu ra*. Ví dụ, một tầng ẩn với m nơ-ron nhận vector đầu vào $x \in \mathbb{R}^n$ và tạo ra vector $h \in \mathbb{R}^m$: $h = f(Wx + b)$. Ở đây, h là một hàm vector với m đầu ra, mỗi đầu ra phụ thuộc vào tất cả n đầu vào. Trong trường hợp này, đạo hàm không còn là một vector mà là một **ma trận Jacobian** [?, p.23]. Với hàm $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$, Jacobian là một ma trận kích thước $m \times n$, trong đó phần tử ở hàng i , cột j là đạo hàm riêng của đầu ra thứ i theo đầu vào thứ j :

$$\frac{\partial f}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}, \quad \text{với} \quad \left(\frac{\partial f}{\partial x} \right)_{ij} = \frac{\partial f_i}{\partial x_j}.$$

Ma trận Jacobian là nền tảng để áp dụng quy tắc chuỗi trong không gian đa chiều.

2.2 Quy tắc Chuỗi với Ma trận Jacobian

Trong giải tích một biến, quy tắc chuỗi cho phép tính đạo hàm của hàm hợp: nếu $z = g(y)$ và $y = h(x)$ thì $\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$. Trong không gian đa chiều, quy tắc chuỗi được mở rộng một cách tự nhiên: nếu $z = g(y)$ với $y \in \mathbb{R}^m$ và $y = h(x)$ với $x \in \mathbb{R}^n$, thì Jacobian của hàm hợp $z(x) = g(h(x))$ là *tích của hai ma trận Jacobian* [?, p.24]:

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x},$$

trong đó $\frac{\partial z}{\partial y}$ có kích thước (số đầu ra của z) $\times m$, $\frac{\partial y}{\partial x}$ có kích thước $m \times n$, và tích của chúng có kích thước (số đầu ra của z) $\times n$. Như vậy, chúng ta có thể "xâu chuỗi" các Jacobian qua các lớp của mạng.

2.3 Ví dụ: Jacobian của Hàm Kích hoạt Element-wise

Một ví dụ quan trọng là Jacobian của hàm kích hoạt áp dụng theo từng phần tử (element-wise) $h = f(z)$, với $h, z \in \mathbb{R}^n$ và $h_i = f(z_i)$. Vì mỗi đầu ra h_i chỉ phụ thuộc duy nhất vào đầu vào z_i , Jacobian là một ma trận đường chéo [?, p.25]:

$$\frac{\partial h}{\partial z} = \begin{bmatrix} f'(z_1) & 0 & \cdots & 0 \\ 0 & f'(z_2) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & f'(z_n) \end{bmatrix} = \text{diag}(f'(z)).$$

Điều này đặc biệt hữu ích vì nó cho phép chúng ta biểu diễn phép nhân với Jacobian này như một phép nhân theo từng phần tử (Hadamard product) với vector $f'(z)$.

2.4 Các Jacobian Cơ bản Thường Gặp

Trong quá trình lan truyền ngược, chúng ta thường xuyên cần tính Jacobian của các phép toán tuyến tính cơ bản. Dưới đây là một số công thức quan trọng [?, p.32]:

1. **Phép biến đổi affine:** $z = Wx + b$, với $W \in \mathbb{R}^{m \times n}$, $x \in \mathbb{R}^n$, $b \in \mathbb{R}^m$. Khi đó:

$$\frac{\partial z}{\partial x} = W \in \mathbb{R}^{m \times n},$$

$$\frac{\partial z}{\partial b} = I_m \quad (\text{ma trận đơn vị kích thước } m \times m).$$

2. **Tích vô hướng:** $s = u^T h$, với $u, h \in \mathbb{R}^m$. Đạo hàm của s theo u là:

$$\frac{\partial s}{\partial u} = h^T \quad (\text{một vector hàng}).$$

Tuy nhiên, theo quy ước hình dạng (sẽ đề cập sau), chúng ta thường coi gradient là vector cột, do đó $\nabla_u s = h$.

Những công thức cơ bản này, cùng với quy tắc chuỗi, cho phép chúng ta tính gradient của bất kỳ hàm mất mát nào đối với tất cả các tham số của mạng.

2.5 Quy ước Hình dạng (Shape Convention) và Sự Khác Biệt với Jacobian

Một điểm cần lưu ý là trong lý thuyết ma trận, Jacobian của một hàm $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ có kích thước $m \times n$. Tuy nhiên, khi triển khai các thuật toán học máy, chúng ta thường áp dụng một **quy ước hình dạng (shape convention)**: gradient của một đại lượng vô hướng đối với một tham số phải có cùng hình dạng (shape) với tham số đó [?, p.50]. Ví dụ, nếu tham số b là một vector cột $\mathbb{R}^m$, thì gradient $\frac{\partial J}{\partial b}$ cũng phải là một vector cột, mặc dù Jacobian $\frac{\partial z}{\partial b}$ là ma trận đơn vị I_m (có thể được xem là một tập hợp các vector cột). Sự khác biệt này nảy sinh vì Jacobian được định nghĩa theo quy ước "đầu ra x đầu vào", trong khi trong tối ưu hóa, chúng ta cần cập nhật tham số theo cùng hình dạng của nó.

Do đó, trong thực hành, chúng ta có hai cách tiếp cận [?, p.55]:

1. Sử dụng Jacobian để tính toán và *chuyển đổi hình dạng* (reshape) ở cuối để phù hợp với tham số. Ví dụ, với $s = u^T h$, Jacobian $\frac{\partial s}{\partial b} = u^T \text{diag}(f'(z))$ là một vector hàng, nhưng để cập nhật b , ta sẽ lấy transpose để có vector cột.
2. Luôn tuân theo quy ước hình dạng: xác định kích thước của các đại lượng và sử dụng phép chuyển vị hoặc sắp xếp lại các số hạng để đảm bảo gradient có cùng hình dạng với tham số. Trong cách này, chúng ta sử dụng khái niệm "tín hiệu lỗi" δ (upstream gradient) có cùng kích thước với tầng ẩn tương ứng.

Ví dụ, với mô hình một tầng ẩn: $z = Wx + b$, $h = f(z)$, $s = u^T h$. Khi đó, gradient của s theo b (theo quy ước hình dạng) là:

$$\frac{\partial s}{\partial b} = \delta^T, \quad \text{trong đó } \delta = (u \circ f'(z)) \in \mathbb{R}^m,$$

có kích thước $1 \times m$ (nếu b là cột, ta sẽ lấy transpose). Tương tự, gradient theo W là:

$$\frac{\partial s}{\partial W} = \delta^T x^T \in \mathbb{R}^{m \times n},$$

khớp với kích thước của W [?, p.51]. Công thức này có thể được hiểu là tích ngoài của tín hiệu lỗi và đầu vào, một dạng rất phổ biến trong lan truyền ngược.

Tóm lại, việc nắm vững ma trận Jacobian và quy tắc chuỗi là chìa khóa để hiểu sâu về lan truyền ngược. Đồng thời, việc áp dụng quy ước hình dạng giúp chúng ta triển khai các phép toán này một cách chính xác và hiệu quả trong các thư viện học sâu. Phần tiếp theo sẽ trình bày chi tiết thuật toán lan truyền ngược (backpropagation) dựa trên những nền tảng này.

3 Thuật toán Lan truyền ngược (Backpropagation)

Trong phần trước, chúng ta đã thiết lập các công cụ toán học cần thiết, bao gồm ma trận Jacobian và quy tắc chuỗi đa biến, để tính gradient của hàm mất mát theo các tham số của mạng. Tuy nhiên, việc áp dụng trực tiếp quy tắc chuỗi cho từng tham số một sẽ dẫn đến sự trùng lặp tính toán rất lớn, đặc biệt khi mạng có nhiều lớp và hàng triệu tham số. **Thuật toán lan truyền ngược (Backpropagation)** là một kỹ thuật tối ưu hóa cho phép tính tất cả các gradient một cách hiệu quả bằng cách tái sử dụng các đạo hàm đã tính ở các tầng trên để tính cho các tầng dưới [?, p.56]. Về bản chất, backpropagation chính là việc áp dụng quy tắc chuỗi một cách có hệ thống trên một **biểu đồ tính toán (computation graph)**, trong đó các gradient được truyền ngược từ đầu ra về đầu vào thông qua các phép toán trung gian. Phần này sẽ trình bày chi tiết cơ chế hoạt động của backpropagation, từ biểu đồ tính toán, lan truyền xuôi, lan truyền ngược, cho đến cách triển khai trong các framework hiện đại.

3.1 Biểu đồ Tính toán (Computation Graph)

Một mạng nơ-ron có thể được biểu diễn dưới dạng một **biểu đồ tính toán có hướng** (directed acyclic graph – DAG), trong đó [?, p.57]:

- **Các nút nguồn (source nodes):** Đầu vào của mạng (ví dụ: x , W , b).
- **Các nút trung gian (interior nodes):** Các phép toán (ví dụ: phép nhân ma trận $z = Wx + b$, hàm kích hoạt $h = f(z)$).
- **Các cạnh:** Truyền kết quả của phép toán từ nút này sang nút khác.
- **Nút đầu ra:** Hàm mất mát J (hoặc điểm số s).

Ví dụ, với mạng một tầng ẩn $s = u^T f(Wx + b)$, biểu đồ tính toán có thể được vẽ như Hình 1 trong slide [?, p.57]. Mỗi nút thực hiện một phép toán cụ thể và chuyển kết quả của nó đến các nút tiếp theo.

3.2 Lan truyền Xuôi (Forward Propagation)

Lan truyền xuôi là quá trình tính giá trị của các nút theo thứ tự từ các nút nguồn đến nút đầu ra, theo thứ tự sắp xếp tô-pô (topological sort) [?, p.59]. Cụ thể, với mỗi nút, ta tính giá trị đầu ra của nó dựa trên các giá trị đầu vào từ các nút tiền nhiệm. Quá trình này được thực hiện cho đến khi đạt được giá trị của hàm mất mát J . Trong quá trình này, các giá trị trung gian (ví dụ: z , h) được lưu lại để sử dụng cho lan truyền ngược sau này.

3.3 Lan truyền Ngược (Backward Propagation)

Lan truyền ngược là quá trình tính gradient của hàm mất mát theo từng nút, bắt đầu từ nút đầu ra và đi ngược về các nút nguồn [?, p.60]. Quy tắc cốt lõi của backpropagation tại mỗi nút là:

$$[\text{Downstream gradient}] = [\text{Upstream gradient}] \times [\text{Local gradient}].$$

Ở đây:

- **Upstream gradient:** Gradient của hàm mất mát theo đầu ra của nút hiện tại (đã được tính từ các nút phía sau).
- **Local gradient:** Jacobian của đầu ra nút hiện tại theo đầu vào của nó (có thể tính toán cục bộ dựa trên phép toán của nút).
- **Downstream gradient:** Gradient của hàm mất mát theo đầu vào của nút hiện tại, sẽ được truyền tiếp về các nút trước.

3.3.1 Nút đơn (Single Node)

Xét một nút đơn giản thực hiện phép biến đổi $h = f(z)$. Nếu upstream gradient tại đầu ra của nút là $\frac{\partial J}{\partial h}$, thì downstream gradient tại đầu vào z được tính là [?, p.62]:

$$\frac{\partial J}{\partial z} = \frac{\partial J}{\partial h} \cdot \frac{\partial h}{\partial z}.$$

Đối với hàm kích hoạt element-wise $h = f(z)$, local gradient là ma trận đường chéo $\text{diag}(f'(z))$, vì vậy phép nhân trở thành tích Hadamard: $\frac{\partial J}{\partial z} = \frac{\partial J}{\partial h} \circ f'(z)$.

3.3.2 Nút với nhiều đầu vào (Multiple Inputs)

Khi một nút có nhiều đầu vào (ví dụ: $z = Wx + b$), nó có nhiều local gradient, mỗi cái tương ứng với một đầu vào [?, p.66]. Ví dụ:

- $\frac{\partial z}{\partial W} = x^T$ (theo quy ước hình dạng).
- $\frac{\partial z}{\partial x} = W$.
- $\frac{\partial z}{\partial b} = I$.

Sau khi tính downstream gradient cho từng đầu vào, chúng được truyền tiếp đến các nút tương ứng.

3.3.3 Nút rẽ nhánh (Branching Nodes)

Khi một giá trị được sử dụng làm đầu vào cho nhiều nút khác nhau (rẽ nhánh), gradient từ các nhánh sẽ được **cộng dồn** (sum) tại điểm rẽ nhánh [?, p.67]. Nguyên lý này xuất phát từ quy tắc chuỗi: nếu một biến x đóng góp vào nhiều hàm khác nhau $y_1, y_2, \dots, y_n$, thì đạo hàm của hàm mất mát J theo x là tổng các đạo hàm riêng qua từng nhánh:

$$\frac{\partial J}{\partial x} = \sum_{i=1}^n \frac{\partial J}{\partial y_i} \frac{\partial y_i}{\partial x}.$$

3.4 Trực giác về các Phép toán Cơ bản

Một số phép toán cơ bản có các quy tắc truyền gradient trực quan sau [?, p.69]:

- **Phép cộng (+):** "Phân phối" (distribute) upstream gradient một cách nguyên vẹn đến tất cả các đầu vào. Ví dụ, nếu $a = x + y$, thì $\frac{\partial J}{\partial x} = \frac{\partial J}{\partial a}$ và $\frac{\partial J}{\partial y} = \frac{\partial J}{\partial a}$.
- **Phép max:** "Định tuyến" (route) upstream gradient đến đúng đầu vào có giá trị lớn nhất. Ví dụ, nếu $b = \max(y, z)$, thì gradient sẽ được truyền đến y nếu $y > z$, và đến z nếu $z > y$.
- **Phép nhân (*):** "Chuyển đổi" (switch) upstream gradient bằng cách nhân với giá trị của đầu vào kia. Ví dụ, nếu $c = a \cdot b$, thì $\frac{\partial J}{\partial a} = \frac{\partial J}{\partial c} \cdot b$ và $\frac{\partial J}{\partial b} = \frac{\partial J}{\partial c} \cdot a$.

Những trực giác này rất hữu ích khi thực hiện backpropagation bằng tay hoặc khi gỡ lỗi các implementation.

3.5 Tính Hiệu quả và Tái sử dụng Kết quả

Một trong những điểm mạnh của backpropagation là khả năng **tái sử dụng các kết quả trung gian**. Thay vì tính từng gradient một cách độc lập (dẫn đến trùng lặp tính toán), backpropagation tính gradient của tất cả các tham số trong một lần duyệt ngược, sử dụng các giá trị đã tính ở các tầng trên để tính cho các tầng dưới [?, p.48]. Cụ thể, nếu ta định nghĩa tín hiệu lỗi (error signal) $\delta^{(k)}$ tại tầng k là gradient của hàm mất mát theo đầu vào pre-activation $z^{(k)}$, thì:

$$\delta^{(k)} = f'(z^{(k)}) \circ \left(W^{(k)T} \delta^{(k+1)} \right),$$

và gradient theo trọng số của tầng k là:

$$\frac{\partial J}{\partial W^{(k)}} = \delta^{(k+1)} a^{(k)T}.$$

Nhờ đó, độ phức tạp tính toán của backpropagation là $O(\text{số tham số})$, tương đương với lan truyền xuôi, thay vì $O(\text{số tham số}^2)$ nếu tính riêng lẻ [?, p.75].

3.6 Giải thuật Backpropagation Tổng quát

Thuật toán backpropagation có thể được tóm tắt qua hai pha [?, p.75]:

1. **Lan truyền xuôi (Forward pass):** Duyệt các nút theo thứ tự tô-pô, tính giá trị của từng nút và lưu các giá trị trung gian.
2. **Lan truyền ngược (Backward pass):**
 - Khởi tạo upstream gradient tại nút đầu ra là $\frac{\partial J}{\partial J} = 1$.
 - Duyệt các nút theo thứ tự tô-pô ngược.
 - Tại mỗi nút, tính downstream gradient bằng cách nhân upstream gradient với local gradient.
 - Cộng dồn gradient nếu có nhiều nhánh.
 - Lưu gradient của các tham số (nếu nút đó chứa tham số) để cập nhật sau.

3.7 Triển khai trong Thực tế và Tự động Phân biệt (Automatic Differentiation)

Trong các framework học sâu hiện đại như PyTorch, TensorFlow, hay JAX, backpropagation được tự động hóa thông qua kỹ thuật **tự động phân biệt (automatic differentiation – autodiff)** [?, p.76]. Mỗi phép toán cơ bản (cộng, nhân, softmax, v.v.) đều có sẵn hàm forward (tính đầu ra) và hàm backward (tính local gradient). Khi xây dựng biểu đồ tính toán, framework sẽ tự động xây dựng đồ thị ngược và thực hiện lan truyền ngược khi cần. Điều này giải phóng các nhà nghiên cứu khỏi việc tính toán thủ công các đạo hàm phức tạp, nhưng vẫn đòi hỏi họ hiểu rõ cơ chế hoạt động để gỡ lỗi và tối ưu hóa [?, p.80].

Một ví dụ mã giả cho lớp ComputationalGraph trong Python [?, p.77]:

```
class ComputationalGraph:
    def forward(inputs):
        # Duyệt các nút theo thứ tự tô-pô
        for gate in self.graph.nodes_topologically_sorted():
            gate.forward()
        return loss # đầu ra của nút cuối cùng

    def backward():
        # Duyệt các nút theo thứ tự ngược
```



```

for gate in reversed(self.graph.nodes_topologically_sorted()):
    gate.backward() # áp dụng quy tắc chuỗi cục bộ
return inputs_gradients

```

Tóm lại, backpropagation là thuật toán nền tảng cho việc huấn luyện mạng nơ-ron, cho phép tính gradient của hàm mất mát theo tất cả các tham số một cách hiệu quả thông qua việc tái sử dụng các đạo hàm trung gian trên biểu đồ tính toán. Việc nắm vững cơ chế của backpropagation, từ các phép toán cục bộ đến việc cộng dồn gradient tại các nhánh, là rất quan trọng để có thể xây dựng, gỡ lỗi và cải tiến các mô hình học sâu, ngay cả khi các framework hiện đại đã tự động hóa phần lớn quá trình này.

4 Kỹ thuật Thực hành và Tối ưu hóa

Sau khi đã nắm vững nền tảng toán học của mạng nơ-ron và thuật toán lan truyền ngược, phần này sẽ trình bày các kỹ thuật thực hành quan trọng để huấn luyện mạng nơ-ron một cách hiệu quả và ổn định. Việc huấn luyện một mạng nơ-ron sâu không chỉ đơn thuần là áp dụng backpropagation và gradient descent; nó đòi hỏi sự kết hợp tinh tế của nhiều kỹ thuật, từ kiểm tra gradient, điều chuẩn (regularization), khởi tạo tham số, đến các chiến lược tối ưu hóa thích ứng. Những kỹ thuật này giúp giải quyết các vấn đề thường gặp như quá khớp (overfitting), gradient biến mất (vanishing gradients), hội tụ chậm, hoặc phân kỳ trong quá trình huấn luyện [?, p.8]. Mặc dù các framework hiện đại đã tự động hóa phần lớn các quy trình này, việc hiểu sâu về từng kỹ thuật là rất cần thiết để có thể gỡ lỗi và tối ưu hóa hiệu suất của mô hình.

4.1 Kiểm tra Gradient (Gradient Check)

Kiểm tra gradient là một kỹ thuật quan trọng để xác minh tính chính xác của các đạo hàm giải tích (analytic gradients) được tính toán bằng backpropagation. Ý tưởng là sử dụng **phương pháp xấp xỉ sai phân trung tâm (centered difference)** để ước lượng gradient bằng số học [?, p.9]:

$$f'(\theta) \approx \frac{J(\theta + \epsilon) - J(\theta - \epsilon)}{2\epsilon},$$

trong đó ϵ là một số rất nhỏ (thường khoảng 10^{-5}). Phương pháp này có sai số tỷ lệ với ϵ^2 , do đó chính xác hơn so với sai phân một phía [?, p.9].

Để thực hiện, với mỗi tham số θ_i :

1. Tăng nhẹ θ_i lên $+\epsilon$, giữ nguyên các tham số khác, thực hiện một forward pass để tính $J(\theta + \epsilon)$.
2. Giảm nhẹ θ_i xuống $-\epsilon$, thực hiện một forward pass để tính $J(\theta - \epsilon)$.
3. Tính gradient số học theo công thức trên.
4. So sánh với gradient giải tích thu được từ backpropagation (thường yêu cầu sai số tương đối nhỏ hơn 10^{-5}).

Tuy nhiên, gradient check có nhược điểm lớn là cực kỳ chậm: với mỗi tham số, cần thực hiện *hai* forward pass. Vì các mạng nơ-ron hiện đại có thể có hàng triệu tham số, gradient check chỉ được sử dụng để gỡ lỗi các layer hoặc các phép toán mới, hoặc kiểm tra trên một tập con nhỏ các tham số [?, p.9]. Trong thực tế, khi đã xây dựng các thành phần cơ bản (cộng, nhân, softmax, v.v.), ta có thể tin tưởng vào các framework autodiff (PyTorch, TensorFlow) và chỉ sử dụng gradient check khi triển khai các layer tùy chỉnh.

Một lưu ý quan trọng khi so sánh gradient số học và giải tích là cần đảm bảo chúng có *cùng hình dạng* (shape). Nếu gradient giải tích được tính theo quy ước Jacobian (dạng hàng), cần chuyển đổi để khớp với hình dạng của tham số trước khi so sánh [?, p.54].

4.2 Điều chuẩn (Regularization)

Mạng nơ-ron có khả năng biểu diễn rất cao, do đó dễ bị **quá khớp (overfitting)** – khi mô hình học quá tốt trên dữ liệu huấn luyện nhưng không thể tổng quát hóa sang dữ liệu mới. Để giảm thiểu,

chúng ta sử dụng các kỹ thuật điều chuẩn [?, p.11].

4.2.1 Điều chuẩn L2 (L2 Regularization / Weight Decay)

L2 regularization thêm vào hàm mất mát một đại lượng tỷ lệ với bình phương của các trọng số (Frobenius norm đối với ma trận):

$$J_R = J + \lambda \sum_{i=1}^L \|W^{(i)}\|_F^2,$$

trong đó $\|W\|_F^2 = \sum_{i,j} W_{ij}^2$ là bình phương Frobenius norm, và λ là siêu tham số điều khiển độ mạnh của điều chuẩn [?, p.11]. Điều chuẩn L2 có tác dụng "phạt" các trọng số lớn, buộc mô hình phải phân tán trọng số một cách đồng đều hơn, từ đó giảm độ phức tạp và hạn chế quá khớp. Các bias thường không được điều chuẩn vì chúng không ảnh hưởng đến biên độ của các kích hoạt [?, p.11].

4.2.2 Dropout

Dropout, được đề xuất bởi Srivastava et al., là một kỹ thuật điều chuẩn rất hiệu quả [?, p.12]. Trong quá trình huấn luyện, tại mỗi bước lan truyền xuôi, mỗi nơ-ron trong tầng ẩn được giữ lại (keep alive) với xác suất p (hoặc bị loại bỏ với xác suất $1 - p$). Các nơ-ron bị loại bỏ sẽ không tham gia vào cả forward lẫn backward pass. Điều này tương đương với việc huấn luyện một tập hợp vô số các mạng con (sub-networks) với kiến trúc khác nhau, và gradient tại mỗi bước chỉ cập nhật cho mạng con hiện tại. Trong quá trình kiểm tra (test time), toàn bộ mạng được sử dụng, nhưng đầu ra của mỗi nơ-ron được nhân với p (hoặc đầu ra của mỗi nơ-ron trong quá trình huấn luyện được chia cho p) để đảm bảo kỳ vọng đầu ra giống nhau [?, p.12].

Dropout giúp ngăn chặn các nơ-ron phụ thuộc quá nhiều vào nhau, thúc đẩy mô hình học các đặc trưng đa dạng và mạnh mẽ hơn.

4.3 Khởi tạo Tham số (Parameter Initialization)

Khởi tạo tham số là một bước quan trọng ảnh hưởng lớn đến tốc độ hội tụ và chất lượng cuối cùng của mô hình. Khởi tạo tất cả các trọng số về 0 hoặc các giá trị quá nhỏ/quá lớn đều có thể dẫn đến các vấn đề như đối xứng (symmetry) hoặc gradient bão hòa [?, p.15].

4.3.1 Khởi tạo Ngẫu nhiên Nhỏ (Small Random Initialization)

Phương pháp đơn giản nhất là khởi tạo các trọng số từ phân phối Gaussian hoặc Uniform với giá trị nhỏ (ví dụ: $\mathcal{N}(0, 0.01)$). Phương pháp này thường hoạt động tốt cho các mạng nông nhưng có thể không đủ hiệu quả cho các mạng sâu.

4.3.2 Khởi tạo Xavier / Glorot (Xavier Initialization)

Được đề xuất bởi Glorot và Bengio (2010), Xavier initialization [?, p.15] khởi tạo các trọng số của ma trận $W \in \mathbb{R}^{n^{(l+1)} \times n^{(l)}}$ theo phân phối đều:

$$W \sim \mathcal{U} \left[-\sqrt{\frac{6}{n^{(l)} + n^{(l+1)}}}, \sqrt{\frac{6}{n^{(l)} + n^{(l+1)}}} \right],$$

trong đó $n^{(l)}$ là số đầu vào (fan-in) và $n^{(l+1)}$ là số đầu ra (fan-out) của ma trận. Công thức này nhằm giữ cho phương sai của các kích hoạt và gradient tương đối ổn định qua các tầng, tránh hiện tượng biến mất hoặc bùng nổ gradient [?, p.15]. Các bias thường được khởi tạo bằng 0.

Đối với các hàm kích hoạt như ReLU, có biến thể He initialization (Kaiming He), sử dụng phân phối $\mathcal{N}(0, \sqrt{2/n^{(l)}})$ để phù hợp với đặc tính không đối xứng của ReLU.

4.4 Chiến lược Học (Learning Strategies)

Tốc độ học (learning rate) α là một trong những siêu tham số quan trọng nhất. Nó quyết định mức độ thay đổi của các tham số sau mỗi bước cập nhật [?, p.15].

$$\theta^{\text{new}} = \theta^{\text{old}} - \alpha \nabla_{\theta} J.$$

4.4.1 Lựa chọn Learning Rate

- Nếu learning rate quá lớn: Mô hình có thể dao động hoặc phân kỳ (diverge) vì bước nhảy vượt quá vùng tối ưu cục bộ. - Nếu learning rate quá nhỏ: Quá trình huấn luyện chậm, có thể bị mắc kẹt ở các điểm yên ngựa (saddle points) hoặc không thể hội tụ trong thời gian cho phép.

4.4.2 Giảm tỷ lệ học (Learning Rate Decay / Annealing)

Thay vì dùng learning rate cố định, ta có thể giảm dần learning rate trong quá trình huấn luyện [?, p.16]. Các chiến lược phổ biến:

- **Step decay:** Giảm learning rate theo cấp số nhân sau mỗi k epoch: $\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/k \rfloor}$.
- **Exponential decay:** $\alpha(t) = \alpha_0 e^{-kt}$.
- **Decay theo thời gian:** $\alpha(t) = \frac{\alpha_0 \tau}{\max(t, \tau)}$, trong đó τ là siêu tham số kiểm soát thời điểm bắt đầu giảm.

Annealing giúp mô hình nhanh chóng tiến gần đến điểm tối ưu với learning rate cao, sau đó tinh chỉnh với learning rate thấp để tránh bỏ lỡ cực tiểu tốt nhất.

4.5 Các Phương pháp Tối ưu Thích ứng (Adaptive Optimization Methods)

Ngoài SGD với learning rate tĩnh, các thuật toán tối ưu thích ứng điều chỉnh learning rate cho từng tham số dựa trên lịch sử gradient, giúp tăng tốc độ hội tụ và giảm độ nhạy với siêu tham số [?, p.16].

4.5.1 Momentum

Momentum sử dụng khái niệm "vận tốc" để tích lũy các gradient trước đó, giúp bước cập nhật mượt mà hơn và vượt qua các điểm yên ngựa [?, p.16]. Cập nhật:

$$v = \mu v - \alpha \nabla_{\theta} J, \quad \theta = \theta + v,$$

trong đó μ là hệ số động lượng (thường khoảng 0.9).

4.5.2 AdaGrad

AdaGrad [?, p.17] duy trì một tổng bình phương gradient cho từng tham số và điều chỉnh learning rate tương ứng:

$$\theta_{t,i} = \theta_{t-1,i} - \frac{\alpha}{\sqrt{\sum_{\tau=1}^t g_{\tau,i}^2 + \epsilon}} g_{t,i},$$

trong đó $g_{t,i}$ là gradient tại thời điểm t theo tham số i , và ϵ là hằng số nhỏ để tránh chia cho 0. Ưu điểm: Tham số có gradient nhỏ sẽ nhận learning rate lớn hơn. Nhược điểm: Tổng bình phương gradient tăng dần, làm learning rate giảm về 0, có thể dừng huấn luyện sớm.

4.5.3 RMSProp

RMSProp [?, p.17] khắc phục nhược điểm của AdaGrad bằng cách sử dụng trung bình trượt (moving average) của bình phương gradient:

$$\text{cache} = \beta \cdot \text{cache} + (1 - \beta) \cdot g^2, \quad \theta = \theta - \frac{\alpha}{\sqrt{\text{cache} + \epsilon}} g,$$

với β thường là 0.9 hoặc 0.99. RMSProp giữ cho learning rate không bị suy giảm đơn điệu và hoạt động tốt trong các môi trường không ổn định.

4.5.4 Adam (Adaptive Moment Estimation)

Adam [?, p.17] kết hợp ý tưởng của Momentum (momentum của gradient) và RMSProp (trung bình trượt của bình phương gradient):

$$m = \beta_1 m + (1 - \beta_1)g, \quad v = \beta_2 v + (1 - \beta_2)g^2,$$

$$\hat{m} = \frac{m}{1 - \beta_1^t}, \quad \hat{v} = \frac{v}{1 - \beta_2^t}, \quad \theta = \theta - \frac{\alpha}{\sqrt{\hat{v}} + \epsilon} \hat{m}.$$

Các giá trị mặc định thường là $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. Adam là một trong những thuật toán được sử dụng phổ biến nhất hiện nay nhờ sự ổn định, tốc độ hội tụ nhanh, và ít siêu tham số cần điều chỉnh [?, p.17].

4.6 Tiền xử lý Dữ liệu (Data Preprocessing)

Tiền xử lý dữ liệu là bước quan trọng để đảm bảo các đặc trưng đầu vào có quy mô và phân phối phù hợp, giúp quá trình huấn luyện ổn định hơn [?, p.14].

4.6.1 Trừ Trung bình (Mean Subtraction)

Loại bỏ giá trị trung bình của từng đặc trưng: $x' = x - \mu$. Điều này tập trung dữ liệu về gốc tọa độ, giúp các gradient tránh đi chệch quá xa.

4.6.2 Chuẩn hóa Phương sai (Normalization)

Chia mỗi đặc trưng cho độ lệch chuẩn của nó: $x' = \frac{x - \mu}{\sigma}$. Điều này đảm bảo tất cả các đặc trưng có cùng thang đo, tránh tình trạng một đặc trưng có phương sai lớn chi phối quá trình học.

4.6.3 Whitening

Whitening [?, p.15] là kỹ thuật mạnh hơn, biến đổi dữ liệu để có ma trận hiệp phương sai là ma trận đơn vị (các đặc trưng không tương quan và có phương sai bằng 1). Tuy nhiên, whitening ít phổ biến hơn do chi phí tính toán cao (cần SVD trên toàn bộ dữ liệu) và thường không cần thiết trong các mô hình sâu.

Tóm lại, việc áp dụng thành thạo các kỹ thuật thực hành và tối ưu hóa này là yếu tố quyết định giữa một mô hình học sâu hoạt động tốt và một mô hình thất bại. Trong khi backpropagation cung cấp cơ chế tính gradient, các kỹ thuật như điều chuẩn, khởi tạo, và tối ưu thích ứng giúp điều hướng quá trình huấn luyện một cách hiệu quả và ổn định, đặc biệt khi làm việc với các kiến trúc phức tạp và dữ liệu quy mô lớn. Sự hiểu biết sâu sắc về các thành phần này sẽ giúp ích rất nhiều cho việc gỡ lỗi, tinh chỉnh siêu tham số, và thiết kế các mô hình mới cho các bài toán NLP hiện đại.

5 Tổng kết

Trong bài giảng này, chúng ta đã cùng nhau khám phá nền tảng toán học và cơ chế vận hành của mạng nơ-ron nhân tạo – một trong những công cụ mạnh mẽ nhất của học sâu hiện đại. Hành trình bắt đầu từ việc tìm hiểu cấu trúc của một nơ-ron đơn lẻ, qua việc xây dựng các tầng nơ-ron với các hàm kích hoạt phi tuyến (Sigmoid, Tanh, ReLU, và các biến thể hiện đại như GELU, SwiGLU), cho đến việc tích hợp chúng thành các mô hình phân loại có khả năng học biểu diễn dữ liệu một cách linh hoạt [?, p.1]. Chúng ta đã thấy rằng chính các hàm kích hoạt phi tuyến đã tạo ra sức mạnh biểu diễn vô hạn cho mạng nơ-ron, cho phép chúng xấp xỉ bất kỳ hàm phức tạp nào và tạo ra các ranh giới quyết định phi tuyến mà các mô hình tuyến tính không thể đạt được [?, p.17].

Tiếp theo, chúng ta đã đi sâu vào phép tính ma trận và ma trận Jacobian – công cụ toán học không thể thiếu để mở rộng quy tắc chuỗi truyền thống sang không gian đa chiều. Các công thức Jacobian cơ bản cho phép biến đổi affine, tích vô hướng và hàm kích hoạt element-wise được xác định một cách tường minh [?, p.30]. Đồng thời, chúng ta đã thảo luận về quy ước hình dạng (shape convention) – một quy ước thực tiễn quan trọng, cho phép chúng ta duy trì sự nhất quán giữa lý thuyết Jacobian và các phép cập nhật tham số trong quá trình tối ưu hóa [?, p.50].

Trên cơ sở đó, thuật toán lan truyền ngược (backpropagation) đã được trình bày một cách có hệ thống thông qua biểu đồ tính toán. Chúng ta đã học cách lan truyền xuôi để tính giá trị của các nút và lưu các trung gian, sau đó lan truyền ngược để truyền tín hiệu lỗi (gradient) từ đầu ra trở về các tham số, áp dụng quy tắc $[downstream] = [upstream] \times [local]$ tại mỗi nút [?, p.62]. Nhờ việc tái sử dụng các đạo hàm trung gian, backpropagation có độ phức tạp tính toán tương đương với lan truyền xuôi, giúp việc huấn luyện các mạng với hàng triệu tham số trở nên khả thi [?, p.74]. Chúng ta cũng đã thấy rằng, mặc dù các framework hiện đại (PyTorch, TensorFlow) tự động hóa việc tính gradient, việc hiểu rõ cơ chế bên dưới là rất quan trọng để gỡ lỗi và thiết kế các tầng tùy chỉnh [?, p.76].

Cuối cùng, chúng ta đã điểm qua một loạt các kỹ thuật thực hành không thể thiếu để huấn luyện mạng nơ-ron thành công. Từ kiểm tra gradient (gradient check) để xác minh tính chính xác của đạo hàm, điều chuẩn L2 và dropout để chống quá khớp, khởi tạo Xavier để duy trì phương sai gradient qua các tầng, cho đến các chiến lược học thích ứng như Adam, RMSProp và momentum – tất cả đều đóng góp vào sự ổn định và tốc độ hội tụ của mô hình [?, p.8]. Những kỹ thuật này, cùng với tiền xử lý dữ liệu hợp lý, là những "bí quyết" giúp các mô hình học sâu đạt được hiệu suất vượt trội trong thực tế.

Như vậy, chúng ta đã hoàn thành một vòng tròn kiến thức: từ việc hiểu cách biểu diễn từ vừng (word vectors) trong các bài trước, đến việc xây dựng và huấn luyện các mạng nơ-ron để giải quyết các bài toán phân loại phức tạp như nhận dạng thực thể (NER) [?, p.10]. Những nền tảng này sẽ là bước đệm vững chắc để chúng ta tiếp cận các kiến trúc tiên tiến hơn trong các bài giảng tiếp theo: mạng nơ-ron hồi quy (RNN) để xử lý chuỗi, cơ chế chú ý (attention), và kiến trúc Transformer – xương sống của các mô hình ngôn ngữ lớn hiện nay.

Sự kết hợp giữa lý thuyết toán học khắt khe, thuật toán hiệu quả và các kỹ thuật thực hành sắc sảo chính là chìa khóa để làm chủ học sâu. Dù các framework ngày càng mạnh mẽ, việc thấu hiểu những nguyên lý nền tảng này sẽ luôn là tài sản quý giá của mỗi nhà nghiên cứu và kỹ sư NLP.

References